

Performance Benchmarking of Algorithmic AI with Minesweeper

The purpose of this project is to evaluate whether a more advanced algorithmic AI, which typically achieves a higher win rate, justifies its increased computational cost when compared to a basic algorithmic AI such as a Random Choice AI. In general, more advanced algorithmic approaches are expected to outperform simpler ones, even at the expense of greater computational overhead.

Hypothesis:

It is hypothesised that AI agents with higher algorithmic complexity will achieve higher win rates at the cost of increased computational time. Furthermore, the performance advantage of advanced AI agents is expected to become more pronounced as mine density increases.

To test this assumption, three algorithmic AI agents were implemented to solve an 8×8 Minesweeper game with varying mine densities ranging from 1 to 30 mines. The agents evaluated in this study are:

1. **Random Choice AI ($O(1)$)** – a baseline agent that selects moves uniformly at random.
2. **Deterministic Rule-Based AI ($O(n)^2$)** – an agent that applies fundamental Minesweeper inference rules.
3. **Subset Inference AI ($O_2(n)$)** – an advanced agent that employs subset-based logical reasoning to infer safe moves.

Each agent was evaluated across multiple simulated games, and performance was measured in terms of win rate (accuracy) and average computational time, allowing for a direct comparison between solution quality and computational cost. Each AI agent operated under identical game rules and had access only to information that would be available to a human player during gameplay.

Experimental Procedure

For each mine configuration, each AI agent played multiple independent Minesweeper games. At the start of each game, the board was randomly generated, and the first move was constrained to a non-mine cell to prevent immediate loss. During gameplay, AI agents repeatedly selected moves according to their respective decision-making strategies until either all non-mine cells were revealed (win) or a mine was triggered (loss).

Performance Metrics

Two primary metrics were used to evaluate performance:

- **Win Rate (Accuracy):** The percentage of games successfully completed without triggering a mine.

- **Average Computational Time:** The mean time taken per game, measured in seconds, representing the computational cost of each AI agent.

Results for each configuration were averaged across all simulated games to reduce the impact of randomness and improve statistical reliability.

Data Collection and Analysis

Performance data were recorded for each AI agent at every mine density. The collected data were visualised using line graphs to illustrate trends in win rate and computational cost as mine density increased. These visualisations enabled a direct comparison of algorithmic efficiency and performance trade-offs among the AI agents.

Results and Graph Interpretation

Figure 1.1

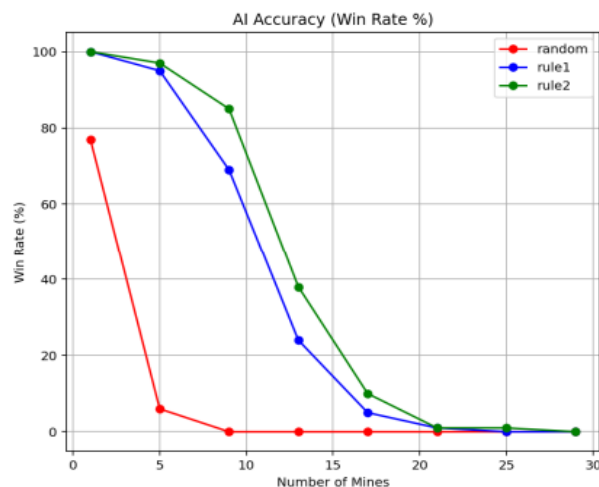
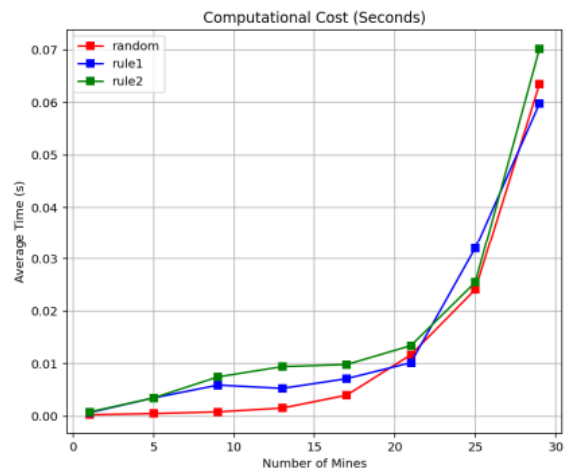


Figure 1.2



Figures 1.1 and 1.2 show the performance of the three AI agents across increasing mine densities on an 8×8 Minesweeper grid. Figure 1.1 presents the **win rate (accuracy)**, while Figure 1.2 illustrates the corresponding **average computational cost**.

Win Rate Analysis (Figure 1.1)

As shown in Figure 1.1, the **Random Choice AI** experiences a steep decline in win rate as the number of mines increases. Even at relatively low mine densities, its performance drops close to zero, demonstrating the limitations of non-deterministic decision-making in complex environments.

The **deterministic rule-based AI** performs significantly better than the random choice AI at low mine counts, achieving high win rates when simple inference rules are sufficient. However, as mine density increases beyond moderate levels, its accuracy decreases rapidly. This indicates that basic inference rules alone are insufficient to handle higher levels of uncertainty.

The **Subset Inference AI** consistently achieves the highest win rate across all mine configurations. While its performance also declines as mine density increases, it maintains a clear advantage over the other two agents, particularly in the mid-range mine counts. This demonstrates the effectiveness of subset-based logical reasoning in identifying safe moves that simpler algorithms fail to detect.

Computational Cost Analysis (Figure 1.2)

Figure 1.2 shows that computational cost increases with algorithmic complexity. The **Random Choice AI** exhibits the lowest computational cost due to its minimal processing requirements. The **deterministic rule-based AI** incurs slightly higher computational cost as it evaluates neighbouring cells and applies inference rules.

The **Subset Inference AI** has the highest computational cost across all mine densities. This increase becomes more pronounced as the number of mines rises, reflecting the additional set comparisons and logical operations required for subset inference. Despite this, the absolute computational cost remains relatively low, indicating that the performance improvements are achieved without excessive computational expense.

Trade-off Between Accuracy and Computational Cost

Together, Figures 1.1 and 1.2 highlight a clear **trade-off between computational cost and performance efficiency**. While simpler AI agents offer faster execution times, they suffer from significantly lower accuracy. In contrast, more advanced algorithmic AI agents require greater computational resources but achieve substantially higher win rates, particularly at moderate mine densities.

These results support the hypothesis that increased algorithmic complexity leads to improved performance at the expense of higher computational cost. Furthermore, the results suggest that the additional computational overhead of the Subset Inference AI is justified by its superior accuracy.

Implementation Details

The Minesweeper environment and AI agents were implemented in Python using a grid-based representation. The simulation was designed to ensure fairness, repeatability, and controlled benchmarking across all AI agents.

Game Environment Representation

The Minesweeper board is represented as two separate 8×8 grids:

- A **visible board**, which stores the information available to the AI agent (hidden cells, revealed numbers, and flagged mines).
- A **backend board**, which stores the ground truth of mine locations and adjacent mine counts.

This separation ensures that AI agents only operate using information that would be available to a human player during gameplay.

Mine locations are generated randomly for each game, and adjacent mine counts are calculated using a neighbourhood lookup function that identifies all valid surrounding cells.

Neighbor Evaluation and Flood Fill

A helper function is used to compute the valid neighbouring cells for any given board position. When a cell with zero adjacent mines is revealed, a **flood-fill algorithm** is applied to automatically reveal all connected zero-value cells and their borders. This mirrors standard Minesweeper gameplay mechanics and improves efficiency by reducing unnecessary repeated moves.

AI Decision-Making Strategies

Three AI agents of increasing complexity were implemented:

Random Choice AI

The Random Choice AI selects a move uniformly at random from all unrevealed cells. This agent serves as a baseline for comparison, as it performs no inference and has minimal computational overhead.

Deterministic Rule-Based AI

The deterministic rule-based AI applies fundamental Minesweeper logic rules. Specifically, when the number displayed on a revealed cell equals the number of flagged neighbouring cells, all remaining hidden neighbours are inferred to be safe and selected for reveal. Additionally, when the number equals the total number of hidden neighbours plus flagged neighbours, all hidden neighbours are flagged as mines. These rules allow the agent to make safe decisions without guessing when sufficient information is available.

Subset Inference AI

The Subset Inference AI extends deterministic reasoning by performing subset-based logical inference. It compares the hidden neighbour sets of adjacent revealed cells. When one set is a subset of another and both require the same number of mines, the difference between the two sets can be inferred to contain safe cells. This enables the agent to identify safe moves that simpler rule-based methods cannot detect.

Flag Placement Mechanism

Before selecting a move, deterministic and subset-based agents attempt to place flags using logical constraints. This process iterates until no further flags can be placed, ensuring that inference opportunities are maximised before making a decision.

Gameplay Loop and Termination Conditions

Each game proceeds iteratively until one of two conditions is met:

- **Win condition:** All non-mine cells are revealed.
- **Loss condition:** A mine is revealed.

To ensure fairness and prevent trivial losses, the first move is constrained to a cell with zero adjacent mines.

Performance Measurement

For each AI agent and mine configuration, multiple games are simulated. Two metrics are recorded:

- **Win rate (accuracy):** The proportion of games successfully completed.
- **Average computational time:** The mean execution time per game, measured in seconds.

These metrics are averaged across simulations to reduce the impact of randomness and provide reliable benchmarking data.

Typical Algorithm Performance Benchmarking Graphs

Figure 2.1

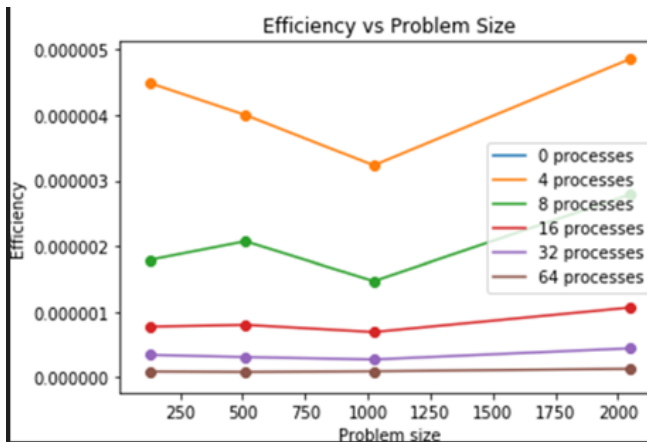
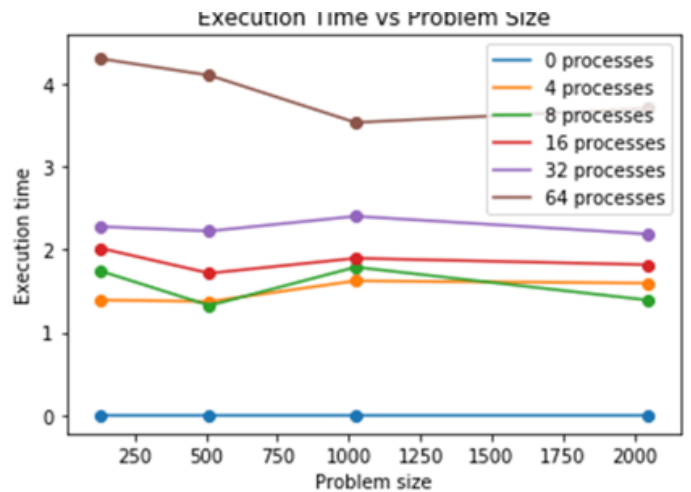


Figure 2.2



Figures 2.1 and 2.2 illustrate the relationship between algorithmic complexity, win rate, and computational cost as the number of mines increases. The results demonstrate that AI agents with higher computational complexity incur greater computational cost; however, this increased cost is accompanied by improved efficiency, reflected by higher win rates.

Figure 2.3

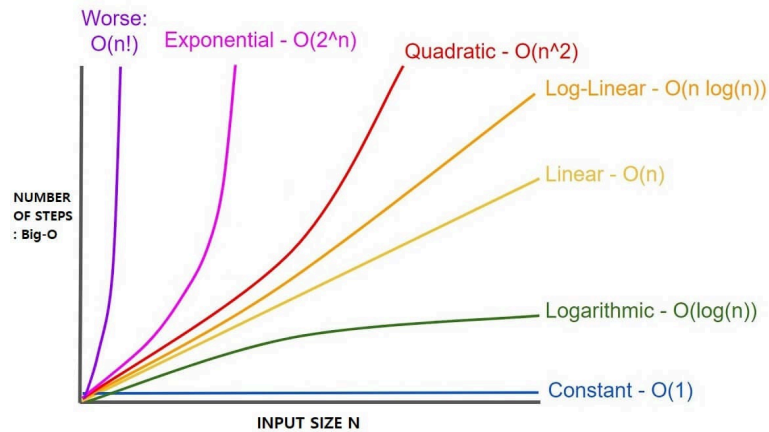


Figure 2.3 illustrates how an algorithm's computational cost grows as the input size (n) increases.

- **$O(1)$ – Constant time runtime** does not change with input size.
- **$O(\log n)$ – Logarithmic** Very efficient growth (e.g., binary search).
- **$O(n)$ – Linear runtime** grows proportionally with input size.
- **$O(n \log n)$ – Log-linear, common** in efficient sorting and structured searches.
- **$O(n^2)$ – Quadratic cost** increases rapidly due to nested operations.
- **$O(2^n)$, $O(n!)$ – Exponential / Factorial:** Extremely expensive; impractical for large inputs.

The steeper the curve, the higher the computational cost as the problem becomes more complex. This theoretical model aligns with the experimental results observed in the Minesweeper AI benchmarking.

Limitations

Although the results provide meaningful insights into the performance trade-offs between algorithmic AI agents, several limitations should be acknowledged.

First, the experiment was conducted exclusively on a fixed **8×8 grid size**. While this allows for controlled and consistent comparisons, the findings may not generalise directly to larger Minesweeper boards, where algorithmic complexity and inference difficulty would increase significantly.

Second, the AI agents implemented in this study rely solely on **deterministic and logical inference**. No probabilistic reasoning was employed when deterministic rules were insufficient. As a result, the AI agents may be forced to make random decisions in ambiguous situations, limiting their performance at higher mine densities.

Third, computational cost was measured using **execution time in Python**, which is influenced by factors such as interpreter overhead and system performance. Consequently, the measured runtimes may not accurately reflect true algorithmic efficiency in lower-level or optimised implementations.

Additionally, the **first move was constrained to a safe cell**, which eliminates early losses but slightly deviates from standard Minesweeper gameplay. While this approach ensures fair comparison across AI agents, it may inflate overall win rates.

Finally, the experiment did not account for **memory usage or scalability beyond time complexity**. Future studies could include additional metrics such as memory consumption and performance on larger boards to provide a more comprehensive evaluation.

Conclusion

This project investigated whether increased algorithmic complexity in AI agents justifies the associated rise in computational cost when solving the Minesweeper problem. Three AI agents of varying complexity were evaluated: a Random Choice AI, a Deterministic Rule-Based AI, and a Subset Inference AI.

The experimental results demonstrate a clear relationship between algorithmic complexity, accuracy, and computational cost. The Random Choice AI, while computationally efficient, performed poorly as mine density increased, highlighting the limitations of non-inferential approaches. The deterministic rule-based AI achieved improved accuracy at low to moderate mine densities but struggled as the problem became more complex. In contrast, the Subset Inference AI consistently achieved the highest win rate across all configurations, particularly in scenarios with higher mine density.

Although the Subset Inference AI incurred greater computational cost, the increase remained relatively small in absolute terms. This indicates that the additional computational overhead is justified by the significant improvement in performance. These findings support the hypothesis that more advanced algorithmic AI systems provide superior efficiency at the expense of higher computational cost.

Overall, the results confirm that in constraint-based problem domains such as Minesweeper, increased algorithmic sophistication leads to more effective decision-making. This study highlights the importance of balancing computational efficiency with solution accuracy when designing AI systems and provides a foundation for further exploration into probabilistic and hybrid inference methods.

Reference

 Using A.I. to DESTROY Minesweeper

 I coded minesweeper using python print console version

 CS50 AI Minesweeper - Project 1B | Complete Walkthrough + Solution